

DAY 3

Python + LLM APIs for AI Automation

Insurance Helpdesk Bot Backend using Python, OpenAI-compatible APIs and FastAPI

Duration	4 Hours / Half Day
Audience	Beginner to Intermediate UiPath, RPA and Automation Developers
Domain	Insurance Claims, Policy Servicing, Helpdesk and Underwriting Support
Mode	Instructor-led Hands-on Lab
Outcome	Build an Insurance Helpdesk Bot backend using Python and LLM APIs

Training Positioning

Day 1 built Python automation foundations. Day 2 built prompt engineering capability. Day 3 connects Python with real LLM APIs and exposes AI capability as services that UiPath can consume.

1. Day 3 Learning Objectives

- Use OpenAI SDK and OpenAI-compatible API endpoints from Python.
- Configure API keys safely using environment variables and .env files.
- Build insurance-specific LLM functions for claim summary, policy Q&A and classification.
- Parse and validate LLM output into JSON for UiPath workflows.
- Handle common API errors using retries, logging and defensive coding.
- Expose an AI claim-assistance service using FastAPI.

2. 4-Hour Session Flow

Time	Topic	Hands-on Outcome
25 min	LLM API architecture for automation teams	Understand Python -> LLM -> UiPath flow
35 min	API keys, .env and safe configuration	Create secure client configuration
40 min	Claim summarization API client	Generate claim summary from raw notes
35 min	Policy Q&A function	Answer coverage questions safely
35 min	Claim classification labels	Classify Health / Vehicle / Life / Property claims
35 min	JSON parsing and validation	Create UiPath-ready structured output
35 min	Error handling, retries and logging	Make API usage production-friendly
40 min	FastAPI endpoint	Expose Insurance Helpdesk Bot backend

3. Concept: OpenAI SDK and OpenAI-Compatible API Usage

Concept Explanation: The OpenAI Python SDK provides a simple way to call chat and text-generation models. Some providers, such as OpenRouter, expose OpenAI-compatible endpoints. This means the same OpenAI client can be used with a different base URL and model name.

Insurance Automation Example

A claims team receives raw claim notes. Python sends the text to an LLM and receives a claim summary, risk label or JSON response. UiPath can then consume the output and continue the workflow.

Basic API Client Pattern

```
from dotenv import load_dotenv
from openai import OpenAI
import os

load_dotenv()

client = OpenAI(
    api_key=os.getenv("OPENAI_API_KEY")
)

completion = client.chat.completions.create(
    model="gpt-4.1-mini",
    messages=[
        {"role": "system", "content": "You are an insurance automation assistant."},
        {"role": "user", "content": "Summarize this insurance claim note."}
    ],
    temperature=0.2
)

print(completion.choices[0].message.content)
```

Trainer Note

If participants are using OpenRouter, keep the OpenAI SDK but set `base_url="https://openrouter.ai/api/v1"` and use a supported OpenRouter model name such as `openai/gpt-4.1-mini`.

OpenRouter-Compatible Example

```
from dotenv import load_dotenv
from openai import OpenAI
import os

load_dotenv()

client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=os.getenv("OPENAI_API_KEY")
)

completion = client.chat.completions.create(
    model="openai/gpt-4.1-mini",
    messages=[
        {"role": "system", "content": "You are an insurance automation assistant."},
        {"role": "user", "content": "Summarize this claim note in 5 bullet points."}
    ],
    temperature=0.2
)

print(completion.choices[0].message.content)
```

4. Concept: Authentication, API Keys and Safe Configuration

Concept Explanation: API keys must not be hard-coded in Python files or notebooks. They should be stored in environment variables, `.env` files for training labs, or enterprise secret stores such as Azure Key Vault, AWS Secrets Manager or UiPath Orchestrator Assets.

.env File for Training Lab

```
OPENAI_API_KEY=your_api_key_here  
OPENAI_BASE_URL=https://openrouter.ai/api/v1  
OPENAI_MODEL=openai/gpt-4.1-mini
```

Reusable Configuration Code

```
from dotenv import load_dotenv  
from openai import OpenAI  
import os  
  
load_dotenv()  
  
API_KEY = os.getenv("OPENAI_API_KEY")  
BASE_URL = os.getenv("OPENAI_BASE_URL", "https://openrouter.ai/api/v1")  
MODEL_NAME = os.getenv("OPENAI_MODEL", "openai/gpt-4.1-mini")  
  
if not API_KEY:  
    raise ValueError("OPENAI_API_KEY is missing. Please add it to your .env file.")  
  
client = OpenAI(  
    api_key=API_KEY,  
    base_url=BASE_URL  
)  
  
print("Client configured successfully")  
print("Model:", MODEL_NAME)
```

Small Assignment 1

Create a .env file. Store OPENAI_API_KEY, OPENAI_BASE_URL and OPENAI_MODEL. Run the configuration code and confirm that no key is printed on screen.

5. Concept: Designing Prompts for API Calls

Concept Explanation: Day 2 focused on prompt design. Day 3 wraps those prompts inside reusable Python functions. This makes prompts easier to test, reuse, version and expose to UiPath.

Reusable LLM Call Function

```
def call_llm(system_prompt, user_prompt, temperature=0.2):
    completion = client.chat.completions.create(
        model=MODEL_NAME,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt}
        ],
        temperature=temperature
    )
    return completion.choices[0].message.content
```

Why This Matters for UiPath Developers

A reusable function is like a reusable UiPath workflow. Instead of copying API code everywhere, create one tested function and call it for claim summary, policy Q&A, classification and JSON extraction.

6. Hands-on Lab 1: Build a Claim Summarization API Client

Business Scenario

A claim handler receives raw claim notes from an email, call center system or document-processing workflow. The AI service must create a short operational summary.

Input Claim Note

```
claim_note = """
```

Customer Rajesh Sharma submitted a health insurance claim on 12 June 2026. He was admitted to City Care Hospital for dengue fever treatment. Total bill amount is INR 85,000. Policy number is POL1001.

Documents submitted include hospital bill, discharge summary and doctor prescription.

Claim is currently pending document verification.

"""

Step-by-Step Code

```
system_prompt = """
```

```
You are an insurance claim summarization assistant.
```

```
Use factual, concise and professional language.
```

```
Do not approve or reject the claim.
```

```
"""
```

```
user_prompt = f"""
```

```
Summarize the following claim note.
```

Output format:

- Customer Name
- Policy Number
- Claim Type
- Claim Amount
- Documents Submitted
- Current Status
- 3-Line Summary

Claim Note:

```
{claim_note}
```

```
"""
```

```
summary = call_llm(system_prompt, user_prompt)
```

```
print(summary)
```

Expected Output

Customer Name: Rajesh Sharma

Policy Number: POL1001

Claim Type: Health Insurance / Hospitalization

Claim Amount: INR 85,000

Documents Submitted: Hospital bill, discharge summary, doctor prescription

Current Status: Pending document verification

Summary: Rajesh Sharma submitted a health claim for dengue treatment at City Care Hospital. The claim amount is INR 85,000. Submitted documents are awaiting verification.

Mini Assignment 2

Modify the prompt to create three summary styles: customer-friendly summary, claim manager summary and audit summary.

7. Hands-on Lab 2: Create a Policy Q&A Function

Concept Explanation

Policy Q&A helps users ask coverage-related questions. The AI should answer only from the supplied policy text. This is an important enterprise guardrail.

Policy Text and Question

```
policy_text = """
Health Secure Plus covers hospitalization expenses up to INR 5,00,000.
Room rent is covered up to INR 5,000 per day.
Pre-hospitalization expenses are covered for 30 days.
Post-hospitalization expenses are covered for 60 days.
Cosmetic treatment and experimental treatment are excluded.
"""
```

```
question = "Does this policy cover post-hospitalization expenses?"
```

Reusable Function

```
def answer_policy_question(policy_text, question):
    system_prompt = """
    You are an insurance policy Q&A assistant.
    Answer only using the supplied policy text.
    If the answer is not found, say: Not available in supplied policy text.
    """

    user_prompt = f"""
```


Policy Text:

{policy_text}

Customer Question:

{question}

Provide a short answer in simple language.

"""

return call_llm(system_prompt, user_prompt)

answer = answer_policy_question(policy_text, question)

print(answer)

Mini Assignment 3

Test the function with three questions: room rent limit, cosmetic treatment coverage and ambulance coverage. Observe when the model should say not available in supplied policy text.

8. Hands-on Lab 3: Generate Classification Labels for Claims

Concept Explanation

Classification helps route claims to the right UiPath queue, team or workflow. The output should be controlled and limited to accepted labels.

Allowed Labels

- Health Claim
- Vehicle Claim
- Life Claim
- Property Claim
- Travel Claim
- Unknown

Classification Code

claim_description = """

Customer reported vehicle damage after a road accident.

Repair estimate is INR 2,40,000. Police report is not attached.

"""

```
def classify_claim(claim_description):
```

```
    system_prompt = """
```

```
    You are an insurance claim classification assistant.
```

```
    Return only one label from this list:
```

```
    Health Claim, Vehicle Claim, Life Claim, Property Claim, Travel Claim, Unknown.
```

```
    """
```

```
    user_prompt = f"""
```

```
    Classify the following claim description:
```

```
    {claim_description}
```

```
    """
```

```
    return call_llm(system_prompt, user_prompt)
```

```
label = classify_claim(claim_description)
```

```
print("Claim Label:", label)
```

Mini Assignment 4

Create five claim descriptions and classify them. Add a confidence field manually: High, Medium or Low.

9. Concept: Structured Output and JSON Response Design

Concept Explanation: UiPath workflows need predictable output. JSON helps automation developers parse values, assign variables, add queue items, update systems and trigger downstream decisions.

JSON Prompt Pattern

```
json_prompt = f"""
```

```
Extract claim information from the text below.
```

```
Return valid JSON only. Do not add explanation outside JSON.
```

Required JSON fields:

- customer_name
- policy_number
- claim_type
- claim_amount
- submitted_documents
- current_status
- next_action

Claim Note:

{claim_note}

"""

LLM Call

```
json_text = call_llm(  
    "You are an insurance automation assistant. Return valid JSON only.",  
    json_prompt  
)  
  
print(json_text)
```

10. Hands-on Lab 4: Parse LLM Output into JSON

Concept Explanation

LLM output should not be trusted blindly. Even when we ask for JSON, we must parse and validate it before sending it to UiPath.

```
import json  
  
def parse_json_response(json_text):  
    try:  
        return json.loads(json_text)  
    except json.JSONDecodeError as error:  
        print("Invalid JSON returned by model")  
        print("Error:", error)  
        return None  
  
parsed_output = parse_json_response(json_text)
```

```
print(parsed_output)
```

Validation Function

```
required_fields = [
    "customer_name",
    "policy_number",
    "claim_type",
    "claim_amount",
    "submitted_documents",
    "current_status",
    "next_action"
]

def validate_claim_json(data):
    errors = []

    if data is None:
        return ["No JSON data available"]

    for field in required_fields:
        if field not in data:
            errors.append(f"Missing field: {field}")

    if "submitted_documents" in data and not
    isinstance(data["submitted_documents"], list):
        errors.append("submitted_documents must be a list")

    return errors

errors = validate_claim_json(parsed_output)

if errors:
    print("Validation failed:")
    for error in errors:
        print("-", error)
else:
```

```
print("JSON is valid for UiPath consumption")
```

Mini Assignment 5

Add validation for `claim_amount`. It should be numeric or a clean string that can be converted into a number.

11. Concept: Response Parsing and Validation

- Always define the expected response format before calling the model.
- Use JSON when output will go to UiPath, APIs or databases.
- Validate required fields before downstream processing.
- Reject or retry when output is invalid.
- Log original input, cleaned output and validation status for audit.

Enterprise Guardrail

LLM output is useful, but automation should verify it. In insurance, trust is important; validation is how we keep trust operational.

12. Concept: Error Handling, Retries and Logging

Concept Explanation: LLM API calls can fail because of network issues, rate limits, invalid API keys, model unavailability or invalid responses. Production code should retry safely and log errors clearly.

Safe API Call with Retry

```
import time
```

```
import logging
```

```
logging.basicConfig(  
    filename="insurance_ai_api.log",  
    level=logging.INFO,  
    format="%(asctime)s - %(levelname)s - %(message)s"  
)
```

```
def call_llm_with_retry(system_prompt, user_prompt, retries=3, delay=2):
```

```
for attempt in range(1, retries + 1):
    try:
        logging.info(f"Calling LLM API. Attempt {attempt}")
        result = call_llm(system_prompt, user_prompt)
        logging.info("LLM API call successful")
        return result
    except Exception as error:
        logging.error(f"LLM API call failed on attempt {attempt}: {error}")
        if attempt == retries:
            raise
        time.sleep(delay)

# Example usage
# result = call_llm_with_retry(system_prompt, user_prompt)
```

Mini Assignment 6

Change the API key temporarily to an invalid value in a test environment and observe the logged error. Restore the correct key immediately after the exercise.

13. Concept: Designing AI Services Consumable by UiPath

Concept Explanation: UiPath can consume Python-based AI services through HTTP requests. FastAPI is a lightweight Python framework for creating APIs.

Target Architecture

- UiPath sends claim text to FastAPI endpoint.
- FastAPI calls Python LLM function.
- LLM returns summary, label or JSON.
- FastAPI validates and sends clean response to UiPath.
- UiPath continues the workflow using structured output.

Simple Architecture

UiPath Robot -> HTTP Request -> FastAPI Service -> LLM API -> JSON Response -> UiPath Queue / Workflow Decision

14. Hands-on Lab 5: Create a FastAPI Endpoint for AI Claim Assistance

Install Required Packages

```
pip install fastapi uvicorn python-dotenv openai
```

Create file: app.py

```
from fastapi import FastAPI
from pydantic import BaseModel
from dotenv import load_dotenv
from openai import OpenAI
import os

load_dotenv()

API_KEY = os.getenv("OPENAI_API_KEY")
BASE_URL = os.getenv("OPENAI_BASE_URL", "https://openrouter.ai/api/v1")
MODEL_NAME = os.getenv("OPENAI_MODEL", "openai/gpt-4.1-mini")

client = OpenAI(api_key=API_KEY, base_url=BASE_URL)

app = FastAPI(title="Insurance Helpdesk Bot API")

class ClaimRequest(BaseModel):
    claim_note: str

class ClaimResponse(BaseModel):
    summary: str

@app.post("/claim-summary", response_model=ClaimResponse)
def claim_summary(request: ClaimRequest):
    system_prompt = """
    You are an insurance claim summarization assistant.
    Use factual language. Do not approve or reject the claim.
    """

    user_prompt = f"""
    Summarize this claim note in 5 bullet points:
    """
```

```
{request.claim_note}
```

```
"""
```

```
completion = client.chat.completions.create(
    model=MODEL_NAME,
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt}
    ],
    temperature=0.2
)

return ClaimResponse(summary=completion.choices[0].message.content)
```

Run the API

```
uvicorn app:app --reload
```

Test API using Python

```
import requests
```

```
payload = {
    "claim_note": "Customer submitted health claim for dengue treatment. Bill amount is INR 85,000. Policy POL1001. Documents submitted: bill and discharge summary."
}
```

```
response = requests.post("http://127.0.0.1:8000/claim-summary", json=payload)
print(response.status_code)
print(response.json())
```

UiPath Connection Tip

In UiPath, use HTTP Request activity with POST method, URL `http://127.0.0.1:8000/claim-summary` and JSON body containing `claim_note`.

15. Extended Lab: FastAPI Endpoint Returning JSON for UiPath

```
class ClaimJsonRequest(BaseModel):
```

```
    claim_note: str
```

```
@app.post("/claim-json")
```

```
def claim_json(request: ClaimJsonRequest):
```

```
    system_prompt = """
```

```
    You are an insurance automation assistant.
```

```
    Return valid JSON only. Do not add explanation outside JSON.
```

```
    """
```

```
    user_prompt = f"""
```

```
    Extract details from the claim note and return JSON with:
```

```
    customer_name, policy_number, claim_type, claim_amount,
```

```
    submitted_documents, current_status, next_action.
```

```
    Claim Note:
```

```
    {request.claim_note}
```

```
    """
```

```
    completion = client.chat.completions.create(
```

```
        model=MODEL_NAME,
```

```
        messages=[
```

```
            {"role": "system", "content": system_prompt},
```

```
            {"role": "user", "content": user_prompt}
```

```
        ],
```

```
        temperature=0.1
```

```
    )
```

```
    raw_output = completion.choices[0].message.content
```

```
    try:
```

```
        return json.loads(raw_output)
```

```
    except Exception:
```

```
        return {
```

```
            "error": "Model did not return valid JSON",
```

```
            "raw_output": raw_output
```

}

Mini Assignment 7

Add one more endpoint: /claim-classification. It should return claim_label and recommended_queue.

16. Image Input Example for Insurance Document Understanding

Concept Explanation: Some models can process image URLs or uploaded images. In insurance, this can support document triage, damaged vehicle image review, receipt understanding or form classification. For Day 3, keep image examples simple and explain that production use requires privacy, consent and compliance review.

OpenRouter-Compatible Image Example

```
completion = client.chat.completions.create(
    model="openai/gpt-4.1-mini",
    messages=[
        {
            "role": "user",
            "content": [
                {
                    "type": "text",
                    "text": "You are an insurance assistant. Describe what is visible in this
image and identify possible insurance relevance."
                },
                {
                    "type": "image_url",
                    "image_url": {
                        "url":
"https://live.staticflickr.com/3851/14825276609_098cac593d_b.jpg"
                    }
                }
            ]
        }
    ]
)
```

)

```
print(completion.choices[0].message.content)
```

Insurance Discussion

Ask participants: Should claim approval be automated from an image alone?

Expected answer: No. Image analysis can support triage, but final decision needs evidence, policy rules and human review.

17. Final Assignment: Insurance Helpdesk Bot Backend

Assignment Objective

Build a Python backend that provides AI assistance for an insurance helpdesk and can be consumed by UiPath through REST APIs.

Required Features

1. Read API key and model configuration from .env file.
2. Create a reusable LLM call function.
3. Create claim summarization function.
4. Create policy Q&A function.
5. Create claim classification function.
6. Create JSON extraction function for UiPath consumption.
7. Add basic error handling and logging.
8. Expose at least three FastAPI endpoints.

Required Endpoints

Endpoint	Input	Output
/claim-summary	claim_note	summary
/policy-qa	policy_text, question	answer
/claim-classification	claim_description	claim_label, recommended_queue
/claim-json	claim_note	structured JSON for UiPath

Evaluation Criteria

- API key is not hard-coded.
- Prompts are insurance-specific and controlled.
- Output is structured wherever required.
- JSON output is validated before use.
- FastAPI endpoints run successfully.
- Code is readable and reusable.
- UiPath integration path is clearly understood.

18. Participant Practice Pack

Practice Exercise A: Claim Summary

Input: A raw hospital claim note. Output: 5 bullet claim summary.

Practice Exercise B: Policy Q&A

Input: Policy coverage paragraph and customer question. Output: short answer based only on supplied policy text.

Practice Exercise C: Claim Classification

Input: Claim description. Output: one label and recommended queue.

Practice Exercise D: UiPath JSON

Input: Claim email text. Output: valid JSON with claim fields and next action.

Practice Exercise E: API Service

Expose all functions through FastAPI and test them using browser docs, Python requests or UiPath HTTP Request activity.

19. Common Mistakes and Fixes

Common Mistake	Recommended Fix
Hard-coding API key in notebook	Use .env file or environment variable
Prompt asks for JSON but model returns explanation	Use: Return valid JSON only. Do not add explanation outside JSON.
No validation before UiPath	Parse JSON and check required fields
Temperature too high	Use 0.1 to 0.3 for enterprise automation
Using broad prompts	Add role, context, rules and output

format

No error logging

Add logging for API failures and validation failures

20. Day 3 Summary

- Participants connected Python with OpenAI-compatible LLM APIs.
- They learned safe API key configuration with .env files.
- They created claim summary, policy Q&A and classification functions.
- They parsed and validated JSON output for UiPath workflows.
- They added error handling, retries and logging concepts.
- They built the foundation for an Insurance Helpdesk Bot backend.

Connection to Day 4

Day 4 can extend this backend with LangChain, RAG, vector databases and insurance knowledge assistants.